

OBJECT ORIENTED PROGRAMMING

Unit-III

GUI Components - AWT package - Layouts – Containers - Event Package - Event Model – Painting – Garbage Collection – Java Applets – Applet Application - Swing Fundamentals - Swing Classes.

PART-A

Two Marks

1. What is event handling?

Event Handling is the mechanism that controls the event and decides what should happen if an event occurs. This mechanism have the code which is known as event handler that is executed when an event occurs. Java Uses the Delegation Event Model to handle the events.

2. Write an applet code to draw smiley.

```
import java.applet.*;
import java.awt.*;

public class Smiley extends Applet {
    public void paint(Graphics g)
    {

        // Oval for face outline
        g.drawOval(80, 70, 150, 150);

        // Ovals for eyes
        // with black color filled
        g.setColor(Color.BLACK);
        g.fillOval(120, 120, 15, 15);
        g.fillOval(170, 120, 15, 15);

        // Arc for the smile
        g.drawArc(130, 180, 50, 20, 180, 180);
    }
}
```

3. Explain about layout.

The layout is used to enhance the look and feel of the application. Layouts use relative positioning to place the components on the container, which means the components automatically adjust their position according to the frame size. The Layout managers enable us to control the way in which visual components are arranged in the GUI forms by determining the size and position of components within

the containers.

4. Write a note on Garbage Collection.

Garbage Collection is process of reclaiming the runtime unused memory automatically. In other words, it is a way to destroy the unused objects. garbage means unreferenced objects. Java garbage collection is the process by which Java programs perform automatic memory management. The garbage collector finds these unused objects and deletes them to free up memory.

5. Compare AWT and Swing(Any two points).

S.NO	Java AWT	Java Swing
1.	AWT components are platform-dependent .	Java swing components are platform-independent .
2.	AWT doesn't support pluggable look and feel .	Swing supports pluggable look and feel .
3.	AWT Components are heavyweight	Swing Components are lightweight

6. What is the purpose of garbage collection in java and when it is used?

Java applications obtain objects in memory as needed. It is the task of garbage collection (GC) in the Java virtual machine (JVM) to automatically determine what memory is no longer being used by a Java application and to recycle this memory for other uses.

7. What is Bytecode?

Bytecode in Java is the reason java is platform-independent, as soon as a Java program is compiled bytecode is generated. To be more precise a Java bytecode is the machine code in the form of a .class file. A bytecode in Java is the instruction set for Java Virtual Machine and acts similar to an assembler.

8. What is event handler?

The EventHandler class provides support for dynamically generating event listeners whose methods execute a simple statement involving an incoming event object and a target object. The EventHandler class is intended to be used by interactive tools, such as application builders, that allow developers to make connections between beans.

9. Differentiate an applet and application.

SNO	Application	Applet
1.	Application program requires a main function	Applet does not requires a

	for its execution.	main function for its execution.
2.	Applications programs that can be execute independently without using the web browser.	Applets are small Java programs that are designed to be included with the HTML web document. They require a Java-enabled web browser for execution.
3.	Java application programs have the full access to the local file system and network.	Applets don't have local disk and network access.

10.What kind of thread is the garbage collector thread?

Daemon thread is a low priority thread that runs in background to perform tasks such as garbage collection. It supports background tasks and only needed while normal threads are executing. It is used in garbage collection mechanism as when the normal threads don't run and remaining threads are daemon threads then the interpreter of the program exits.

11.Give the purpose of containers.

A **Container** class can be described as a special component that can hold the gathering of the component.A container provides a space where a component can be located. A Container in AWT is a component itself and it provides the capability to add a component to itself.Top-Level containers are heavyweight containers such as JFrame, JApplet, JWindow, and JDialog. Low-Level containers are lightweight containers such as JPanel.

12.Give the significance of swing class.

- Provides both additional functionalities and added components to AWT-replacement components
- Swing components are platform-independent.
- Swing components can use a different look and feel.
- Swing components use the Model-View-Controller paradigm (MVC) and thus can provide a much more flexible UI.
- Swing components are lightweight (are less resource-intensive than AWT).

13.Give the syntax of drawImage() and copyArea() methods.

Syntax of drawImage() method:

public abstract boolean drawImage(Image img, int x, int y, ImageObserver observer); is used draw the specified image.

CopyArea():The copyArea() method copies an area from one drawable to another.

Syntax:

```
public void copyArea(int iSource, int iDest, int x, int y, int width, int height, int x2, int y2)
```

14. What are the methods under WindowListener Interface?

- public abstract void windowActivated(WindowEvent e);
- public abstract void windowClosed(WindowEvent e);
- public abstract void windowClosing(WindowEvent e);
- public abstract void windowDeactivated(WindowEvent e);
- public abstract void windowDeiconified(WindowEvent e);
- public abstract void windowIconified(WindowEvent e);
- public abstract void windowOpened(WindowEvent e);

15. How are the java applets used to create applications in internet?

Java applets were small applications written in the Java programming language,.. The applets are **used** to provide interactive features to web applications that Developers can **develop** and debug an applet directly simply by **creating** a Java Virtual Machine of their own. A Java applet could appear in a frame of the web page, a new application window, Sun's AppletViewer, or a stand-alone tool for testing applets.

16. Which class is the top of the AWT event hierarchy?

The java.awt.AWTEvent class is the highest-level class in the AWT event-class hierarchy.

17. What are the uses of references in functions?

The method references can only be used to replace a single method of the lambda expression. A code is more clear and short if one uses a lambda expression rather than using an anonymous class and one can use method reference rather than using a single function lambda expression to achieve the same. In general, one doesn't have to pass arguments to method references.

18. Define Auto Boxing.

Converting a primitive value into an object of the corresponding wrapper class is called autoboxing. For example, converting int to Integer class. The Java compiler applies autoboxing when a primitive value is:

- Passed as a parameter to a method that **expects an object** of the corresponding wrapper class.
- Assigned to a variable of the corresponding **wrapper class**.

19. Define AWT.

Java AWT is also known as Abstract Window Toolkit is an API that is used to develop either GUI or window-based applications in Java. Java AWT components are platform-dependent which implies that they are displayed according to the view of the operating system. It is also heavyweight implying that its components are using the resources of the Operating System. java. awt package provides classes for AWT api. For example, TextField, CheckBox, Choice, Label, TextArea, Radio Button, List, etc

20. list out the listener methods for implementing the KeyListener interface.

- **KeyPressed(KeyEvent e):** This method will be invoked when Key is pressed.
- **KeyReleased(KeyEvent e):** This method will be invoked when a key is released.
- **KeyTyped(KeyEvent e):** This method will be invoked when Key is typed.

21. Swing Component are not thread safe. Why?

- One of the main reason for Java Swing is not thread-safe is to simplify the task of extending its components.
- Another reason for the Java Swing is not thread-safe due to the overhead involved in obtaining and releasing locks and restoring the state.
- Some of the Java Swing component methods will support multi-threaded access like repaint(), revalidate(), and invalidate() methods of JComponent class.

22. What is the relationship between event sources and listeners?

A source is an object that generates an event. This occurs when object changes in some way. Source may generate more than one type of event. A source must register listeners in order for the listeners to receive notifications about a specific type of event. A listener is an object that is notified when an event occurs. A **source** generates an **event** and sends it to one or more **listeners**. **Listener** simply waits until it receives an **event**. Listener processes events and then returns.

23 What is event driven programming?

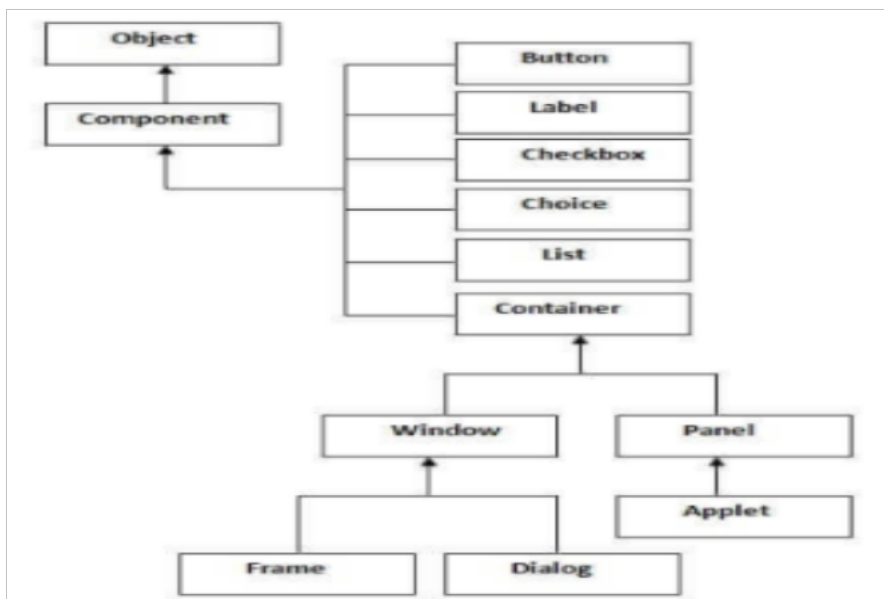
Event-driven programming is when a program is designed to respond to user engagement in various forms. It is known as a programming paradigm in which the flow of program execution is determined by "events." Events are any user interaction, such as a click or key press, in response to prompt from the system. To visualize event programming more clearly, imagine the opposite of event-programming is a software that requires zero user interaction.

PART-B

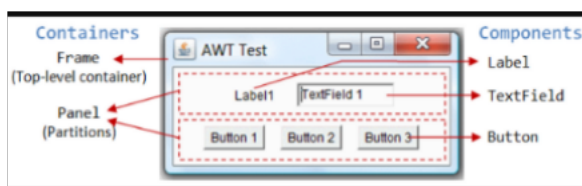
11-Marks

24. What are called GUI components? Explain

- **Java AWT** (Abstract Window Toolkit) is *an API to develop GUI or window-based applications* in java.
- Java AWT components are platform-dependent i.e. components are displayed according to the view of operating system. AWT is heavyweight i.e. its components are using the resources of OS.
- A graphical user interface (GUI) presents a user-friendly mechanism for interacting with an application. A GUI gives an application a distinctive “look and feel.” GUIs are built from GUI components. These are sometimes called controls or widgets—short for window gadgets. A GUI component is an object with which the user interacts via the mouse, the keyboard or another form of input, such as voice recognition.
- Java AWT Hierarchy: **The hierarchy of Java AWT classes are given below.**



AWT Containers and Components



There are two groups of GUI elements:

1. *Component (Widget, Control)*: Components are elementary GUI entities, such as Button, Label, and TextField. They are also called *widgets, controls* in other graphics systems.
2. *Container*: Containers, such as Frame and Panel, are used to *hold components in a specific layout* (such as FlowLayout or GridLayout). A container can also hold sub-containers.

In the above figure, there are three containers: a Frame and two Panels. A Frame is the *top-level container* of an AWT program. A Frame has a title bar (containing an icon, a title, and the minimize/maximize/close buttons), an optional menu bar and the content display area. A Panel is a *rectangular area* used to group related GUI components in a certain layout. In the above figure, the top-level Frame contains two Panels. There are five components: a Label (providing description), a TextField (for users to enter text), and three Buttons (for user to trigger certain programmed actions).

In a GUI program, a component must be kept (or added) in a container. You need to identify a container to hold the components. Every container has a method called `add(Component c)`. A container (say `aContainer`) can invoke `aContainer.add(aComponent)` to add `aComponent` into itself. For example,

```
Panel pnl = new Panel();           // Panel is a container
Button btn = new Button("Press"); // Button is a component
pnl.add(btn);                      // The Panel container adds a Button component
```

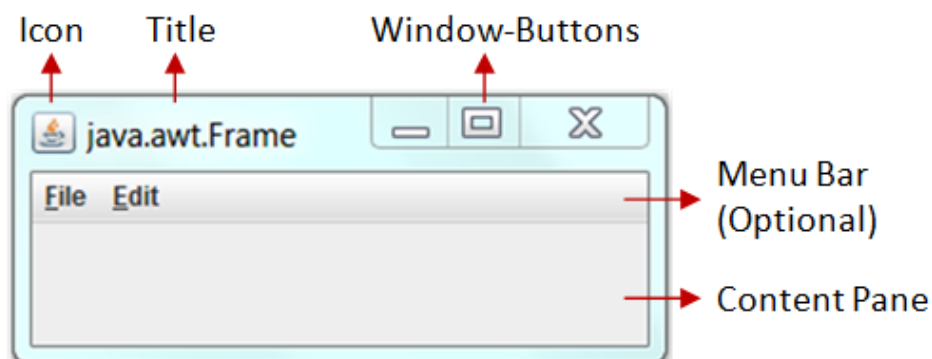
GUI components are also called *controls* (e.g., Microsoft ActiveX Control), *widgets* (e.g., Eclipse's Standard Widget Toolkit, Google Web Toolkit), which allow users to interact with (or control) the application.

AWT Container Classes

Top-Level Containers: Frame, Dialog and Applet

Each GUI program has a *top-level container*. The commonly-used top-level containers in AWT are Frame, Dialog and Applet.

- A Frame provides the "main window" for your GUI application. It has a title bar (containing an icon, a title, the minimize, maximize/restore-down and close buttons), an optional menu bar, and the content display area. To write a GUI program, we typically start with a subclass extending from `java.awt.Frame` to inherit the main window as follows



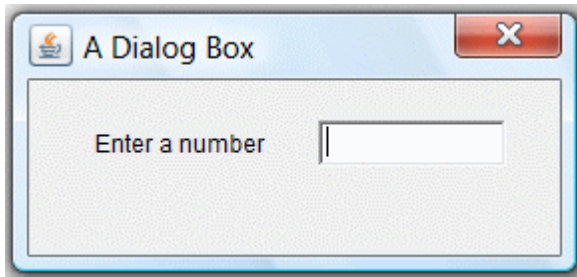
- An AWT Dialog is a "*pop-up window*" used for interacting with the users. A Dialog has a title-bar (containing an icon, a title and a close button) and a content display area, as illustrated.

- An AWT Applet (in package java.applet) is the top-level container for an applet, which is a Java program running inside a browser. Applet is no longer supported in most of the browsers.

Secondary Containers: Panel and ScrollPane

Secondary containers are placed inside a top-level container or another secondary container. AWT provides these secondary containers:

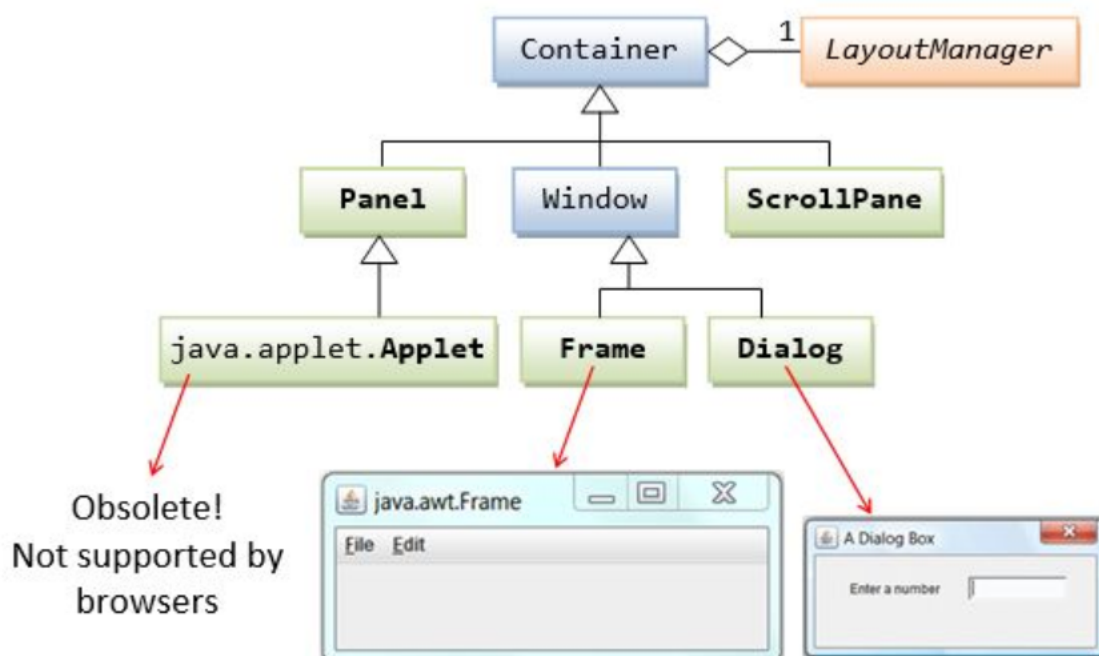
- Panel: a rectangular box used to *layout* a set of related GUI components in pattern such as grid or flow.
- ScrollPane: provides automatic horizontal and/or vertical scrolling for a single child component.
- others.



Hierarchy of the AWT Container Classes

The hierarchy of the AWT Container classes is as follows:

a Container has a LayoutManager to layout the components in a certain pattern, e.g., flow, grid.

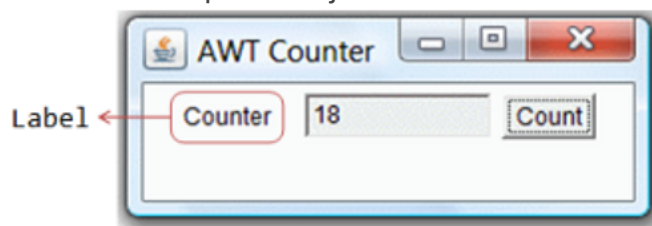


AWT Component Classes

AWT provides many ready-made and reusable GUI components in package `java.awt`. The frequently-used are: Button, TextField, Label, Checkbox, CheckboxGroup (radio buttons), List, and Choice, as illustrated below.



AWT GUI Component: `java.awt.Label`



A `java.awt.Label` provides a descriptive text string. Take note that `System.out.println()` prints to the system console, NOT to the graphics screen. You could use a Label to label another component (such as text field) to provide a text description. Check the JDK API specification for `java.awt.Label`.

AWT Packages

AWT is huge! It consists of 12 packages of 370 classes (Swing is even bigger, with 18 packages of 737 classes as of JDK 8). Fortunately, only 2 packages - `java.awt` and `java.awt.event` - are commonly-used.

1. The `java.awt` package contains the *core* AWT graphics classes:
 - o GUI Component classes, such as Button, TextField, and Label.
 - o GUI Container classes, such as Frame and Panel.
 - o Layout managers, such as FlowLayout, BorderLayout and GridLayout.
 - o Custom graphics classes, such as Graphics, Color and Font.
2. The `java.awt.event` package supports event handling:
 - o Event classes, such as ActionEvent, MouseEvent, KeyEvent and WindowEvent,
 - o EventListenerInterfaces such as ActionListener, MouseListener, MouseMotionListener, KeyListener and WindowListener,
 - o Event Listener Adapter classes, such as MouseAdapter, KeyAdapter, and WindowAdapter.

AWT provides a *platform-independent* and *device-independent* interface to develop graphic programs that runs on all platforms, including Windows, macOS, and Unixes.

25 .Discuss the fundamentals of swing in detail.

Swing in Java is a lightweight GUI toolkit which has a wide variety of widgets for building optimized window based applications. It is a part of the JFC(Java Foundation Classes). It is build on top of the AWT API and entirely written in [java](#). It is platform independent unlike AWT and has lightweight components.

The javax.swing package provides classes for java swing API such as JButton, JTextField, JTextArea, JRadioButton, JCheckbox, JMenu, JColorChooser etc.

Container Class

Any class which has other components in it is called as a container class. For building GUI applications at least one container class is necessary.

Following are the three types of container classes:

1. Panel – It is used to organize components on to a window
2. Frame – A fully functioning window with icons and titles
3. Dialog – It is like a pop up window but not fully functional like the frame

Features of Swing

1. Platform Independent
2. Customizable
3. Extensible
4. Configurable
5. Lightweight
6. Rich Controls
7. Pluggable Look and Feel

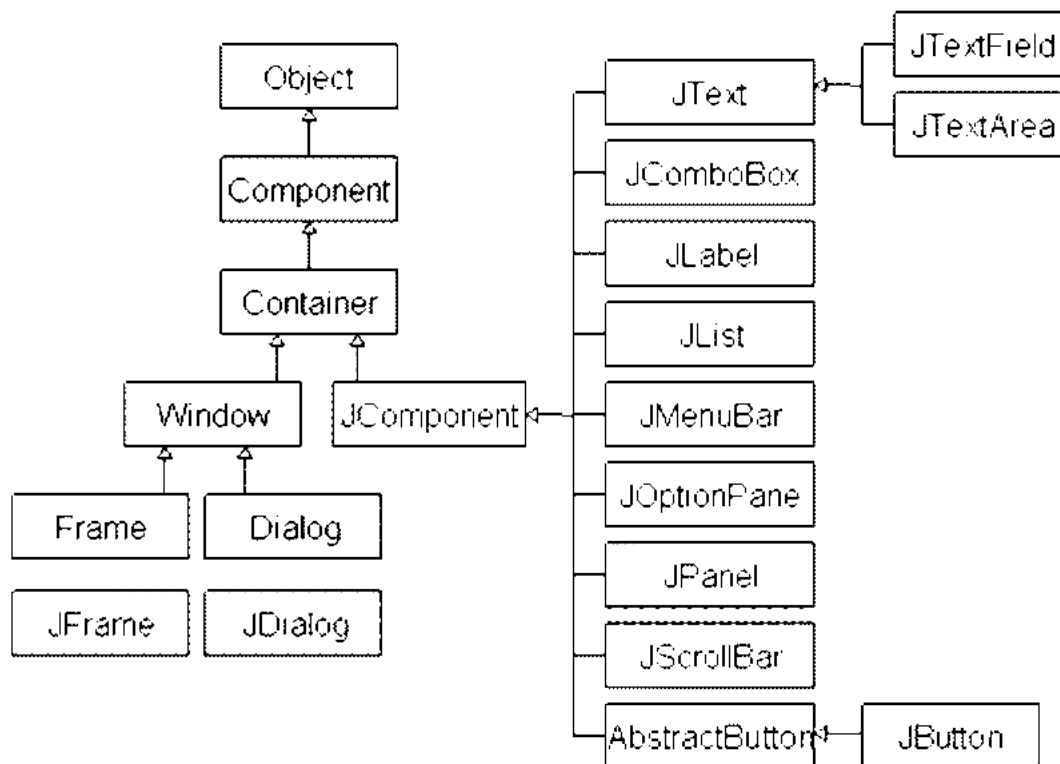
Swing and JFC

JFC is an abbreviation for Java Foundation classes which encompass a group of features for building Graphical User Interfaces(GUI) and adding rich graphical functionalities and interactivity to Java applications. Java Swing is a part of Java Foundation Classes (JFC).

Features of JFC

- Swing GUI components.
- Look and Feel support.
- Java 2D.

AWT and Swing Hierarchy



- **JPanel** : JPanel is Swing's version of AWT class Panel and uses the same default layout, FlowLayout. JPanel is descended directly from JComponent. A panel is an instance of JPanel. A frame can have more than one panels and each panel can have several components. You can also call them parts of Frame. Panels are useful for grouping components and placing them to appropriate locations in a frame.
- **JFrame** : JFrame is Swing's version of Frame and is descended directly from **Frame** class. The component which is added to the **Frame**, is referred as its Content. A frame is an instance of JFrame. Frame is a window that can have title, border, menu, buttons, text fields and several other components. A Swing application must have a frame to have the components added to it.
- **JWindow** : This is Swing's version of Window and has descended directly from **Window** class. Like **Window** it uses BorderLayout by default.
- **JLabel** : JLabel has descended from JComponent, and is used to create text labels. A label is an instance of JLabel class. A label is unselectable text and images. If you want to display a string or an image on a frame, you can do so by using labels. In the above example we wanted to display texts "User" & "Password" just before the text fields, we did this by creating and adding labels to the appropriate positions.
- **JButton** : JButton class provides the functioning of push button. JButton allows an icon, string or both associated with a button.
- **TextField** : TextFields allow editing of a single line of text. Used for capturing user inputs, these are the text boxes where user enters the data.
- **JPasswordField** – Similar to text fields but the entered data gets hidden and displayed as dots on GUI.

Creating a JFrame

There are two ways to create a JFrame Window.

1. By instantiating JFrame class.
2. By extending JFrame class.

Advantages of Swing

Swing has the following advantages over AWT:

- Provides both additional functionalities and added components to AWT-replacement components
- Swing components are platform-independent.
- Swing components can use a different look and feel.
- Swing components use the Model-View-Controller paradigm (MVC) and thus can provide a much more flexible UI.
- Swing components are lightweight (are less resource-intensive than AWT).
- Swing provides built-in double buffering.
- Swing provides paint debugging support for when you build your own components.

Disadvantages of Swing

Swing has a few disadvantages, they are:

- it can be slower than AWT (all components are drawn) as if we're not careful in programming.
- It requires Java 1.2 or a separate JAR file.
- Swing components that look like native components might not act exactly like native components.

26. Explain Applet life cycle and also explain how to provide parameters through HTML with example.

Java Applet

Applet is a special type of program that is embedded in the webpage to generate the dynamic content. It runs inside the browser and works at client side.

Advantage of Applet

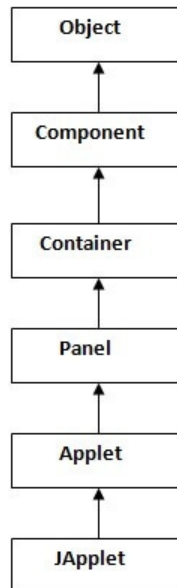
There are many advantages of applet. They are as follows:

- o It works at client side so less response time.
- o Secured
- o It can be executed by browsers running under many platforms, including Linux, Windows, Mac Os etc.

Drawback of Applet

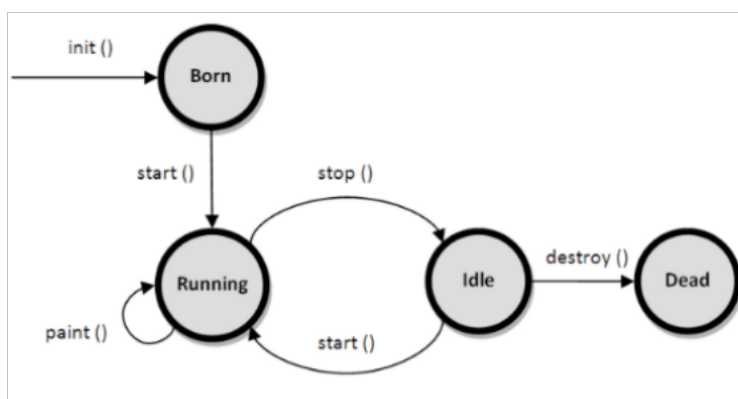
- o Plugin is required at client browser to execute applet.

Hierarchy of Applet



Lifecycle of Java Applet

Applet life cycle defined as how the object created, started, stopped and destroyed during the entire execution of the application is said to applet life cycle. Applet life cycle has 5 methods `init()`, `start()`, `stop()`, `aint()` and `destroy()`. These methods are invoked by the browser to execute. Applet works on the client side so, less processing time.



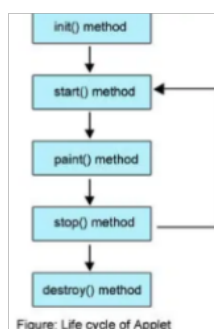
Methods of Applet Life Cycle

Applet life cycle has 5 methods. Methods are `init()`, `start()`, `paint()`, `stop()` and `destroy()`.

- **init():** `init()` method is used to initialize an applet. It is invoking only once at the time

of initialization. Initialized objects are created by the web browser. We can compare this method with a Thread class born state.

- **start():** start() method is used to start the applet. It is invoked after the init() method is invoked. It is invoked each time when browser loading or refreshing. Until init() method is invoked start() method is in inactive state. We can compare this method with Thread class start state
- **stop():** stop() method is used to stop the applet. It is invoked every time when browser stopped or minimized or abrupt failure of the application. After stop() method is called, we can also start() method whenever we want. This method mainly deals with clean up code. We can compare this method with Thread class blocked state
- **destroy():** destroy() method is used to destroy the application once we have done with our applet work. It is invoked only once. Once applet is destroyed we can't start() the applet. We can compare this method with Thread class dead state
- **paint():** paint() method is used for painting any shapes like square, rectangle, trapezium, eclipse, etc. paint() method has parameter as Graphics. This Graphics class gives painting features in an applet. We can compare this method with Thread class runnable state.



Example program that demonstrates the life cycle of an applet is as follows:

To execute the applet by html file, create an applet and compile it. After that create an html file and place the applet code in html file.

```
//First.java
import java.applet.Applet;
import java.awt.Graphics;
public class First extends Applet{

    public void paint(Graphics g){
        g.drawString("welcome",150,150);
    }

}

/*
<applet code="First.class" width="300" height="300">
</applet>
*/
```

27. Discuss the importance of garbage collection and methodology in java.

Garbage Collection in Java is a process by which the programs perform memory management automatically. The Garbage Collector(GC) finds the unused objects and deletes them to reclaim the memory. In Java, dynamic memory allocation of objects is achieved using the new operator that uses some memory and the memory remains allocated until there are references for the use of the object.

When there are no references to an object, it is assumed to be no longer needed, and the memory, occupied by the object can be reclaimed. There is no explicit need to destroy an object as Java handles the de-allocation automatically. The technique that accomplishes this is known as **Garbage Collection**

Garbage Collection is a process of working with the Heap memory and also *mark* or identify the unreachable objects and destroy them with compaction.

Garbage collection in Java is an automatic process and the programmer does not have to explicitly mark objects to be deleted. The implementation mainly lives in the JVM. Each JVM can implement garbage collection. The only requirement is that it should meet the JVM specification.

Types of Garbage Collector

The JVM provides four different garbage collectors, all of them generational. Each one has its own advantages and limitations. The choice of which garbage collector to use lies with the user and there can be numerous differences in the throughput and application pauses.

- **Serial Garbage Collector (GC):** All the garbage collection events are conducted serially in one single thread. Compaction is executed after each garbage collection.
- **Parallel/throughput GC:** Multiple threads are used for minor/small garbage collection. A single thread is used for major garbage collection and Old Generation compaction. Also, the Parallel variant uses multiple threads for major garbage collection and Old Generation compaction.
- **CMS Collector:** Multiple threads are used for small/minor garbage collection using the same algorithm as Parallel. Majority of the garbage collection is multi-threaded, but CMS runs concurrently along with the application processes to minimize application events. No compaction is performed.
- **G1 Collector:** This garbage collector is basically intended as a replacement for CMS. It is parallel and concurrent like CMS, but it works quite differently when it is compared to the older garbage collectors.

Advantages

- The biggest benefit of Java garbage collection is that it automatically **handles the deletion** of unused objects or some objects that are inaccessible to free up memory resources.
- Garbage Collection is now a new standard component of many popular programming languages.
- It makes Java memory-efficient. It is because the garbage collector removes the unreferenced objects from heap memory.
- It is automatically done by the garbage collector which is a part of JVM.

Garbage Collector Running

It is up to the JVM to run the Garbage Collector program. When JVM runs the Garbage Collector, the unreachable objects are destroyed. But still, we cannot guarantee when the JVM will run.

Although we cannot force the GC to execute, we can very well request for a Garbage Collection.

The GC can be requested using any of the following methods.

1) **System.gc():** The System class of Java provides a static method gc () using which we can request the JVM to run Garbage Collector.

2) **Runtime.getRuntime().gc():** Like System.gc (), we can also use the gc () method of "Runtime class" to request JVM to run Garbage Collector.

Note: There is no guarantee that the Garbage Collector will run after a request from

these two methods.

Finalization

Finalization is performed by Garbage Collector just before destroying the objects. As a part of the finalization technique, the Garbage Collector calls the `finalize()` method on the object. The `finalize()` method is used to perform cleanup activities.

The `finalize()` method is provided by the "Object" class and has the following prototype.

protected void finalize () throws Throwable

The `finalize()` method is invoked whenever the object is garbage collected

Note: The Garbage collector only collects the objects that are created using

28. Briefly explain about layout managers in java.

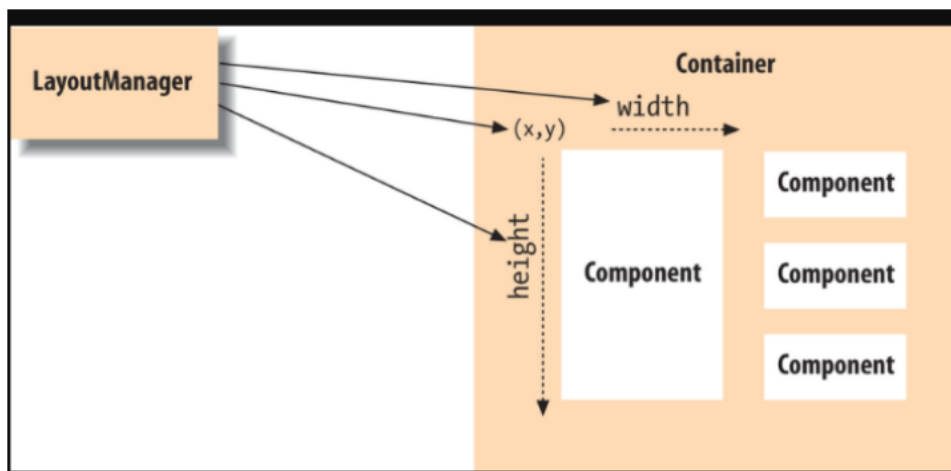
- A layout manager is an object that implements the Layout Manager interface and determines the size and position of the components within a container.
- The Layout Managers are used to arrange components in a particular manner. `LayoutManager` is an interface that is implemented by all the classes of layout managers.
- Layout managers are basically used in the graphics programming of Java. We can develop window-based applications containing various cool layouts.
- The layout manager's job is to fit the components into the available area while maintaining the proper spatial relationships among the components.

Types of LayoutManager

- **FlowLayout:** It arranges the components in a container like the words on a page. It fills the top line from **left to right and top to bottom**. The components are arranged in the order as they are added i.e. first components appears at top left, if the container is not wide enough to display all the components, it is wrapped around the line. Vertical and horizontal gap between components can be controlled. The components can be **left, center or right aligned**.
- **BorderLayout:** It arranges all the components along the edges or the middle of the container i.e. **top, bottom, right and left** edges of the area. The components added to the top or bottom gets its preferred height, but its width will be the width of the container and also the components added to the left or right gets its preferred width, but its height will be the remaining height of the container. The components added to the center gets neither its preferred height or width. It covers the remaining area of the container.
- **GridLayout:** It arranges all the components in a grid of **equally sized cells**, adding them from the **left to right and top to bottom**. Only one component

can be placed in a cell and each region of the grid will have the same size. When the container is resized, all cells are automatically resized. The order of placing the components in a cell is determined as they were added.

- **GridBagLayout:** It is a powerful layout which arranges all the components in a grid of cells and maintains the aspect ratio of the object whenever the container is resized. In this layout, cells may be different in size. It assigns a consistent horizontal and vertical gap among components. It allows us to specify a default alignment for components within the columns or rows.
- **BoxLayout:** It arranges multiple components in either **vertically or horizontally**, but not both. The components are arranged from **left to right or top to bottom**. If the components are aligned **horizontally**, the height of all components will be the same and equal to the largest sized components. If the components are aligned **vertically**, the width of all components will be the same and equal to the largest width components.
- **CardLayout:** It arranges two or more components having the same size. The components are **arranged in a deck**, where all the cards of the same size and the **only top card are visible at any time**. The first component added in the container will be kept at the top of the deck. The default gap at the left, right, top and bottom edges are zero and the card components are displayed either **horizontally or vertically**.



A layout manager at work

- Every container has a default layout manager. When you make a new container, it comes with a LayoutManager object of the appropriate type. You can install a new layout manager at any time by using the `setLayout()` method.

The Abstract Windowing Toolkit (AWT) has the following five layout managers:

- `java.awt.BorderLayout`
- `java.awt.FlowLayout`

- java.awt.GridLayout
- java.awt.CardLayout
- java.awt.GridBagLayout

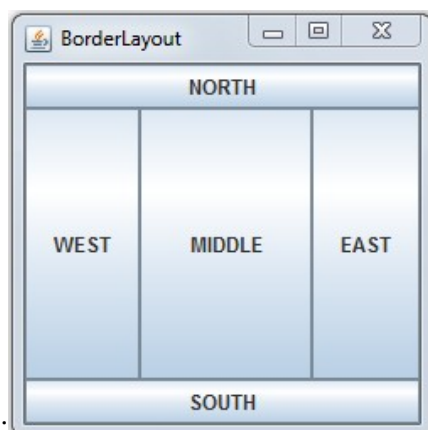
Some of these are used in the swing:

- javax.swing.BoxLayout
- javax.swing.ScrollPanelLayout
- javax.swing.GroupLayout
- javax.swing.SpringLayout

Example: Illustrating the BorderLayout manager

```
import java.awt.*;
import javax.swing.*;
public class BorderLayoutDemo
{
    JFrame frame = new JFrame("BorderLayout");
    JButton b1, b2, b3, b4, b5;
    BorderLayoutDemo()
    {
        b1 = new JButton("NORTH");
        b2 = new JButton("SOUTH");
        b3 = new JButton("EAST");
        b4 = new JButton("WEST");
        b5 = new JButton("MIDDLE");
        frame.add(b1, BorderLayout.NORTH);
        frame.add(b2, BorderLayout.SOUTH);
        frame.add(b3, BorderLayout.EAST);
        frame.add(b4, BorderLayout.WEST);
        frame.add(b5, BorderLayout.CENTER);
        frame.setSize(250, 250);
        frame.setVisible(true);
    }
    public static void main(String args[])
    {
        new BorderLayoutDemo();
    }
}
```

} Output:



29. Elaborately discuss the components of graphics programming

Graphics in any language gives a wonderful look and feel to the users as well as programmers. Programmers draw figures, strings etc with the help of graphics. Without graphics the windows programming is incomplete. Java is not behind. Java provides Abstract Window Toolkit.

- AWT
- Graphics Class
- Drawing Strings
- Drawing Lines
- Drawing Rectangle
- Drawing Ellipses and Circles and Ovals
- Drawing Arcs
- Drawing Polygons

The Graphics class is an abstract class that provides the means to access different graphics devices. It lets us draw images on the screen, display images, and so forth.

It is an abstract class because working with graphics requires detailed knowledge of the platform on which the program runs. The actual work is done by concrete classes that are closely tied to a particular platform.

Once you have a Graphics object, you can call all the methods of the Graphics class without caring about platform specific classes. A Graphics object is always available when you override a component's paint() and update() methods. It is the sole parameter of the Component.paint() and Component.update() methods.

The Graphics class defines a number of drawing functions. Each shape can be drawn edge-only or filled. Objects are drawn and filled in the currently selected graphics color, which is black by default. When a graphics object is drawn that exceeds the dimensions of the window, output is automatically clipped.

Drawing Strings

There are three methods

- drawString ()
- drawChars()
- drawBytes()
- Drawing Lines have only one method drawLine()

Drawing Rectangle Methods

drawRect (),fillRect (),drawRoundRect (),fillRoundRect(),draw3DRect , fill3DRect()

Drawing Ellipses and Circles and Ovals

drawOval (),fillOval()

Drawing Arcs Methods

`drawArc()`, `fillArc()`

Drawing Polygons Methods

`drawPolygon()`, `drawPolyline()`, `fillPolygon()`

Java2D

Java2D is the informal name given to the parts of the Java standard library dealing with drawing and printing 2D lines, shapes, images, gradients, and textures, together with fun options like compositing, filtering, transforming these objects.

Java 3D Graphics

Three-dimensional graphics are not part of the Java standard libraries, but you can use Java OpenGL (known as JOGL), or check out a community project called Java 3D that provides a nice scene-graph API that runs on top of OpenGL.

30.illustrate the basic of event handling in java detail.

Any program that uses GUI (graphical user interface) such as Java application written for windows, is event driven. Event describes the change in state of any object. **For Example** : Pressing a button, Entering a character in Textbox, Clicking or Dragging a mouse, etc.

Components of Event Handling

Event handling has three main components,

- **Events** : An event is a change in state of an object. When you press a button in your program or Android application the state of the button changes from 'Unclicked' to 'Clicked'. This change in the state of our button is called an Event. Events are generated based on how you interact with the GUI. For example- entering some text through the keyboard, moving your cursor, scrolling, etc generates events
- **Events Source** : Event source is an object that generates an event. In Java, nearly everything is an object. The button you press is an object too. Source is the object which generates an event. In other words, a source is an object which undergoes state change. It also provides information about the event to the listener.
- **Listeners** : A listener is an object that listens to the event. A listener gets notified when an event occurs. Listeners are also called as event handlers as they are the ones responsible to handle events occurring at the source. Listeners are interfaces and different types of listeners are used according to the event.

How Events are handled

A source generates an Event and send it to one or more listeners registered with the source. Once event is received by the listener, they process the event and then return. Events are supported by a number of Java packages, like **java.util**, **java.awt** and **java.awt.event**.

Steps involved in event handling

- The User clicks the button and the event is generated.
- Now the object of concerned event class is created automatically and information about the source and the event get populated with in same object.
- Event object is forwarded to the method of registered listener class.
- the method is now get executed and returns.
- Implement appropriate interface in the class.
- Register the component with the listener.